

1. Introduction:

Dans le cadre de la SAE 3, il nous est demandé de concevoir une application web de vote. Cette solution doit être fiable, anonyme et automatisée tout en respectant les différentes phases d'une élection numérique.

L'objectif est de pouvoir remplacer efficacement un vote papier ou téléphonique, plus lourd à organiser et moins fiable.

Dans notre cas, le projet consiste à concevoir et développer une application web permettant le vote en ligne dans différentes catégories musicales. Cette application permet une expérience bien plus fluide et accessible que le vote en présentiel ou par téléphone mais aussi une gestion plus rigoureuse et transparente des propositions, votes et résultats.

2. Contexte du projet:

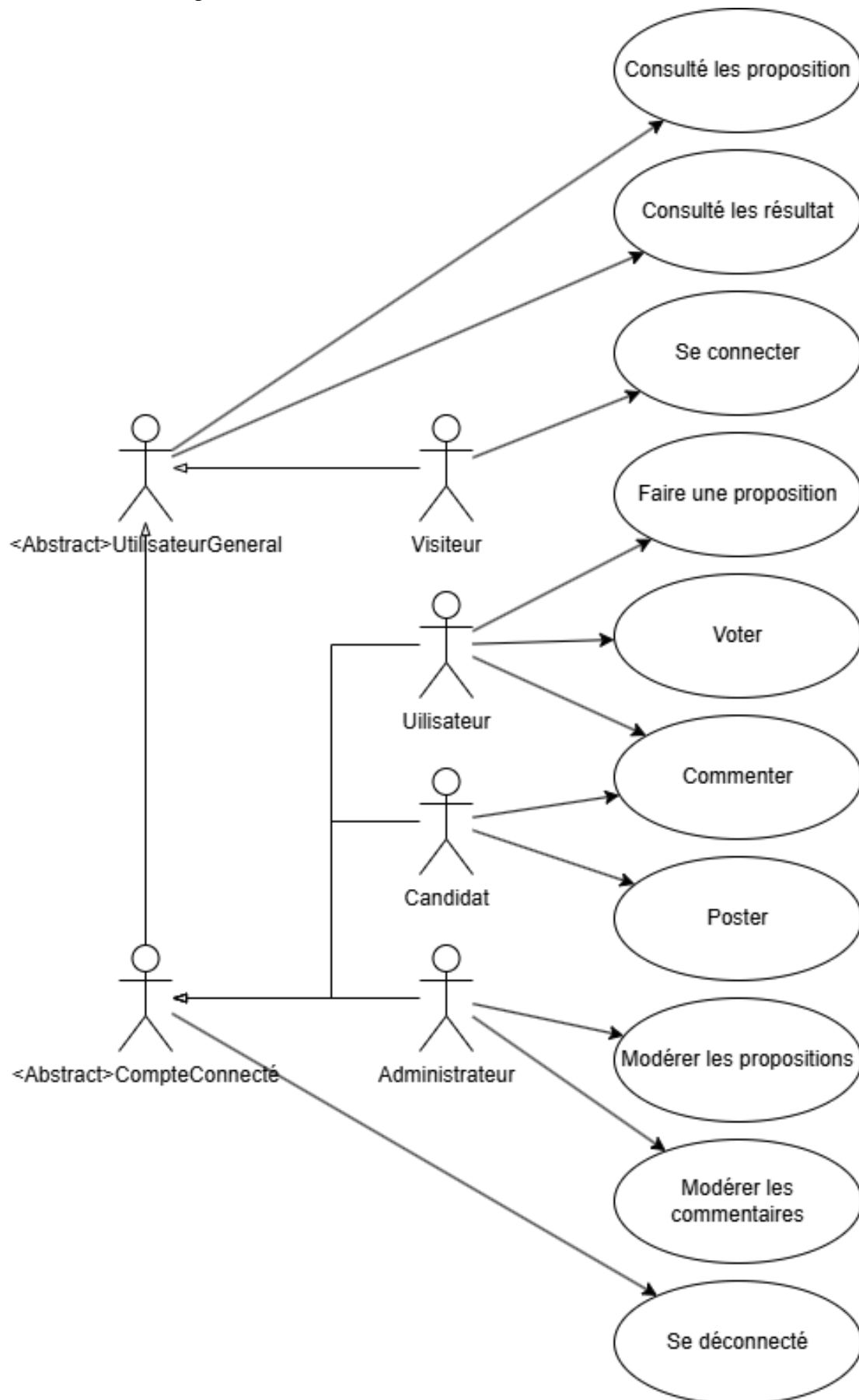
Comme indiqué précédemment, notre application web Toptracks gère le vote d'un concours dans différentes catégories musicales. Ce concours fonctionne par édition, chaque édition peut avoir des catégories disponibles différentes.

L'un des points clé du fonctionnement de Toptracks est le fait que les nominés sont proposés directement par les utilisateurs. Cette approche participative renforce l'engagement du public et permet de créer un concours dynamique, évolutif et représentatif des goûts des votants. L'application doit donc gérer non seulement les votes, mais aussi la soumission, la validation et l'affichage des propositions, tout en garantissant l'anonymat et l'unicité du vote.

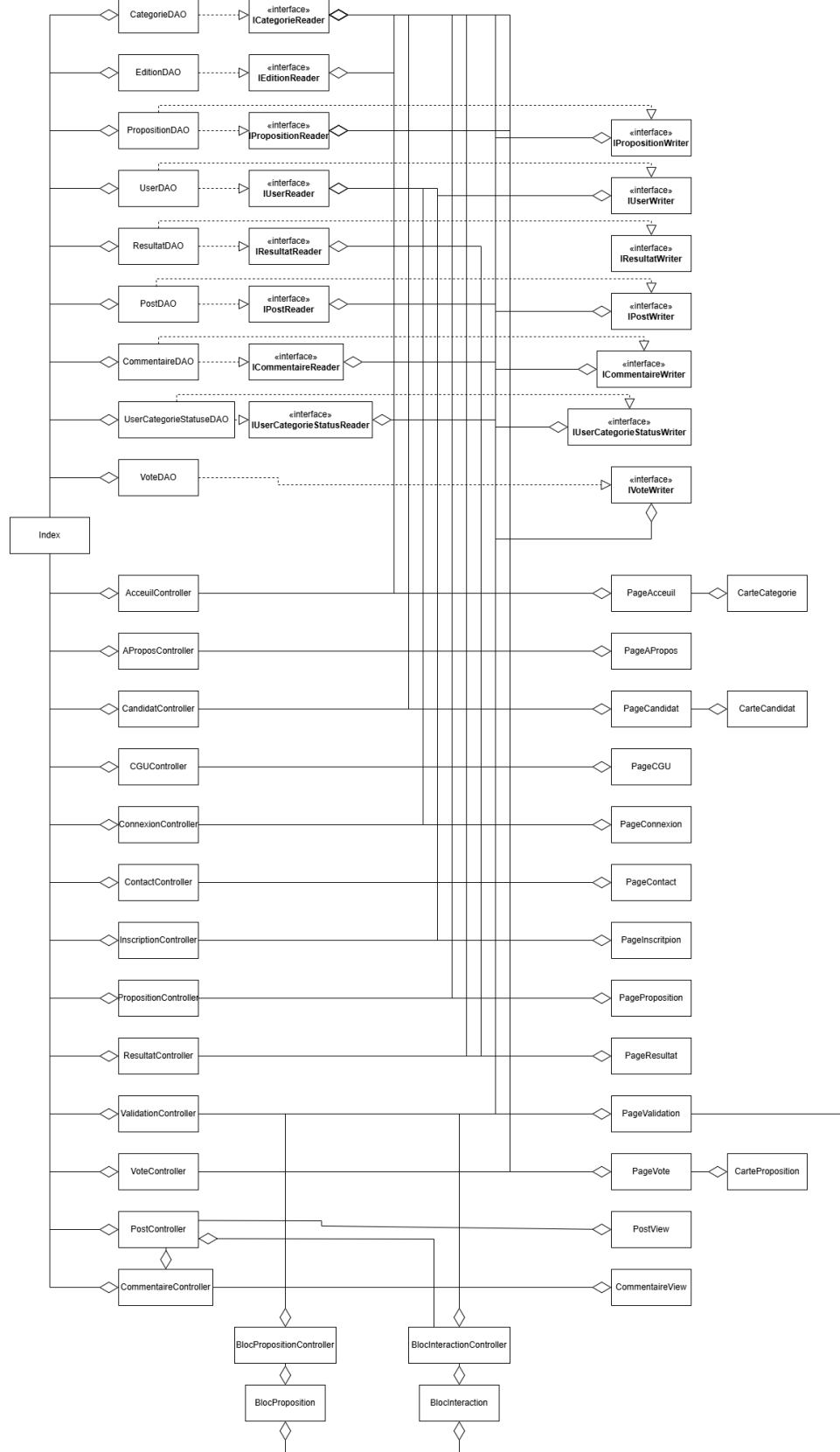
3. Analyse et conception:
- Identification des acteurs

Acteur	Description
Visiteur	Personne non connectée. Il peut consulter les propositions, les catégories, les posts et commentaires, les résultats une fois publiés, ainsi que les pages publiques. Ne peut ni voter, ni commenter, ni proposer.
Utilisateur/Votant	Utilisateur disposant d'un compte il peut effectuer les mêmes actions qu'un visiteur ainsi que faire des propositions voter et commenter.
Candidat	Les candidats ont leur propre espace leur permettant d'accéder au proposition qui leurs sont associés sur lesquelles ils peuvent poster et commenter.
Administrateur	il peut effectuer les mêmes actions qu'un visiteur ainsi que d'écrire des commentaires. Il peut également supprimer des commentaires ou posts(Non implémenter)

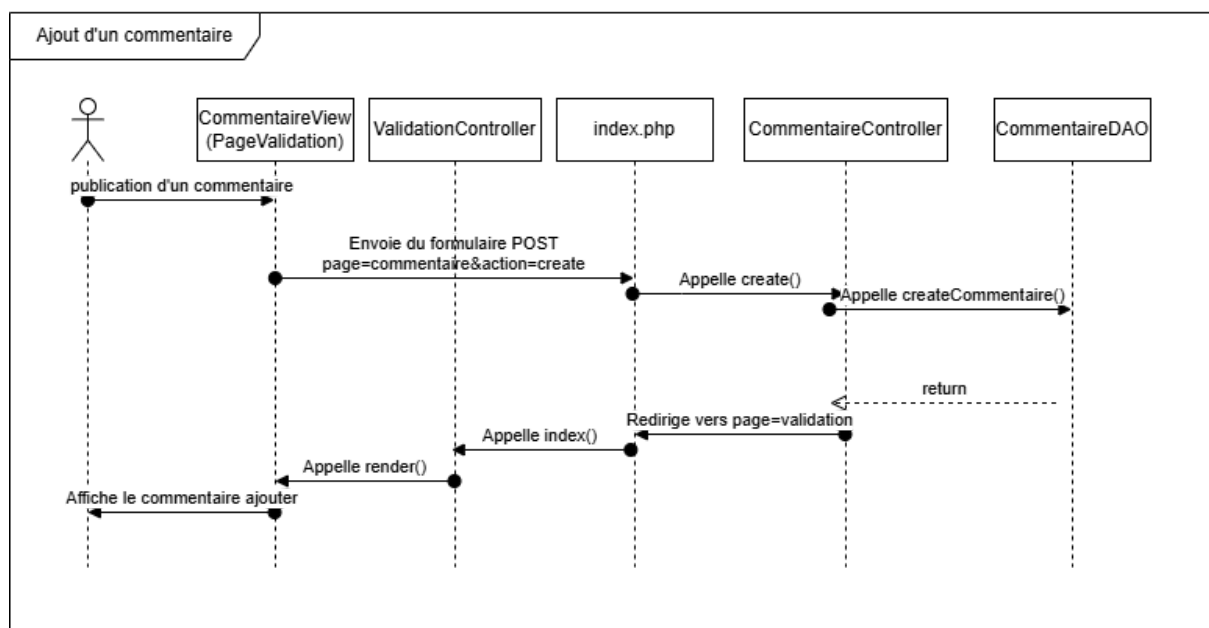
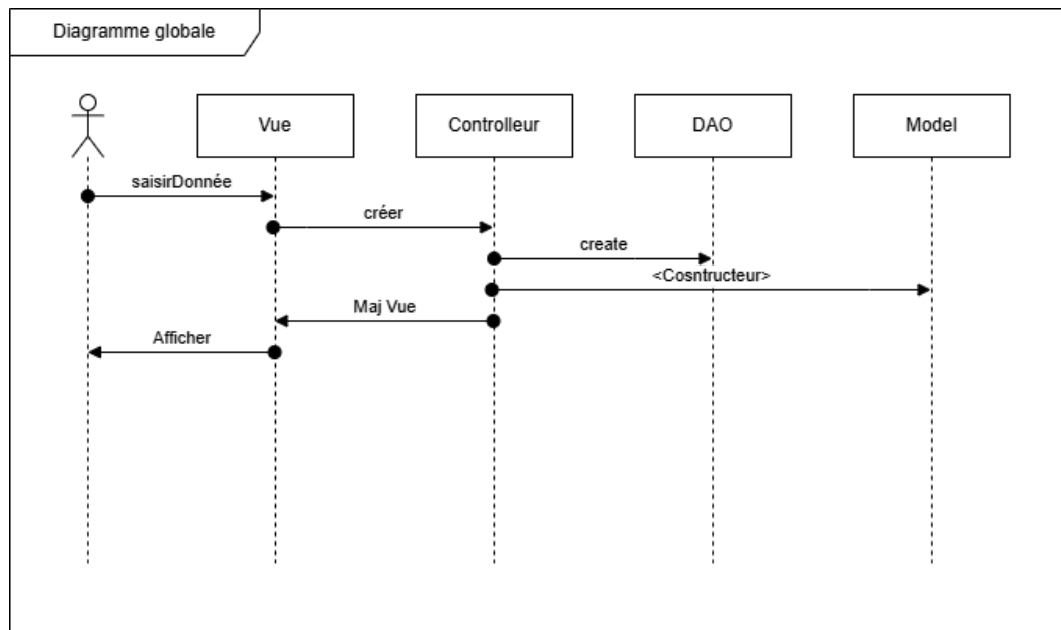
- Diagramme de cas d'utilisation



Certains éléments tels que les services ou encore les modèles ne sont pas présents dans ce diagramme pour le maintenir lisible.



- Diagramme de séquences



- Points conceptuels importants (justification des principes SOLID)

1) SRP: The Single Responsibility Principle

Pour assurer que chaque classe est une responsabilité unique nous avons choisi d'utiliser une architecture MVC (Modèle Vue Contrôleur) complétée par des DAO et services spécialisés.

- Les modèles représentent les données métier et ne contiennent aucune logique.
- Les DAO assurent l'accès aux données en base et les opérations SQL.
- Les contrôleurs orchestrent la logique métier.
- Les vues (pages et composants) se chargent uniquement de l'affichage et de logique basique y étant liés.
- Les services encapsulent des fonctionnalités transversales (connexion BDD, gestion de session, accès API Deezer).
- Index.php assurent le routage de l'application en appelant les contrôleurs adaptés

2) OCP: The Open-Closed Principle

Les classes sont ouvertes à l'extension et fermées à la modification:

- Les DAO peuvent être étendues sans modifier l'existant.
- Les modèles
- Les contrôleurs peuvent également être étendus sans modifier l'existant.
- Les vues sont modulaires grâce au composant réutilisable il est également possible de compléter ces pages sans modifier l'existant.

3) LSP: The Liskov Substitution Principle

Le LSP est respecté grâce à l'utilisation d'interface entre les contrôleurs et DAO. On peut donc modifier notre DAO par exemple pour faire un test unitaire sans avoir à modifier notre contrôleur.

4) ISP : The Interface-Segregation Principle

Les DAO sont séparés en deux interfaces :

IDAORReader

IDAOWriter

Cela évite que les contrôleurs dépendent de méthodes inutiles. Cette séparation réduit le couplage et clarifie les responsabilités.

Une séparation plus fine serait possible mais la complexité des DAO (les modèles ne correspondant pas exactement aux tables SQL) impliquerait un nombre trop important d'interfaces.

5) DIP : The Dependency Inversion Principle

Le DIP recommandent que les modules dépendent d'abstraction plutôt que d'implémentation concrètes. Actuellement les contrôleurs dépendent de certains services et autres contrôleurs. Cela violent légèrement le DIP corriger ce point demanderait des injections de dépendances pour tous les services.

4. Réalisation:

Routeur:

```
session_start();

// Création des dao
$categorieDAO = new CategorieDAO();
$editionDAO = new EditionDAO();
$propositionDAO = new PropositionDAO();
$userDAO = new UserDAO();
$resultatDAO = new ResultatDAO();
$postDAO = new PostDAO();
$commentaireDAO = new CommentaireDAO();
$userCategorieStatusDAO = new UserCategorieStatusDAO();
$voteDAO = new VoteDAO();

// La page par défaut est la page d'accueil
$page = $_GET['page'] ?? 'accueil';

$action = $_GET['action'] ?? 'index';

// gestion des routes en fonction de l'url
switch ($page) {
    case 'accueil':
        if (SessionManager::getInstance()->isCandidat()) {
            header(header: "Location: index.php?page=candidat");
        }
        $controller = new AccueilController(editionReader: $editionDAO,
            categorieReader: $categorieDAO);
        $controller->index();
        break;
    case 'aPropos':
        $controller = new AProposController();
        $controller->index();
        break;
    case 'candidat':
        $controller = new CandidatController(propositionReader: $propositionDAO,
            categorieReader: $categorieDAO);
        $controller->index();
        break;
    case 'cgu':
        $controller = new CGUController();
        $controller->index();
        break;
    case 'connexion':
        $controller = new ConnexionController(userReader: $userDAO);
        if ($action === "login") {
            $controller->login();
        }elseif ($action === 'logout') {
            $controller->logout();
        }
    }
}
```

Le routing de l'application est effectué par mon index.php il lit l'url et appelle le bon contrôleur pour effectuer l'action demandée.

Contrôleur:

```
<?php
namespace controller;

require_once 'vue/page/PageAccueil.php';
require_once 'interfaces/edition/IEditionReader.php';
require_once 'interfaces/categorie/ICategorieReader.php';

use vue\page\PageAccueil;
use service\VotePhase;
use interfaces\edition\IEditionReader;
use interfaces\categorie\ICategorieReader;

2 references | 0 implementations
class AccueilController {
    2 references
    private IEditionReader $editionReader;
    2 references
    private ICategorieReader $categorieReader;

    0 references | 0 overrides
    public function __construct(IEditionReader $editionReader,
    ICategorieReader $categorieReader) {
        $this->editionReader = $editionReader;
        $this->categorieReader = $categorieReader;
    }

    1 reference | 0 overrides
    public function index(): void {
        $phase = VotePhase::getPhaseVote();

        $edition = $this->editionReader->getActive();

        $categories = $this->categorieReader->getCategoriesFromEdition
        (editionId: $edition->getId());

        $page = new PageAccueil(title: "Accueil", phase: $phase,
        categories: $categories);
        $page->render(); // le contrôleur déclenche l'affichage
    }
}
```

Ici on a le contrôleur pour la page d'accueil il fait les appels DAO via les interfaces fournies par le routeur et crée et appelle `render()` de la page d'accueil pour l'afficher.

Interfaces:

```
<?php
namespace interfaces\edition;

require_once 'model/Edition.php';

use model\Edition;

8 references | 1 implementation
interface IEditionReader {
    3 references | 1 override
    public function getActive(): Edition;
    1 reference | 1 override
    public function categorieInActiveEdition(int $categorieId): bool;
}
```

On a un exemple d'interface IEditionReader elle permet d'accéder aux fonctions de lecture de EditionDAO.

DAO:

```
18 references | 9 implementations
abstract class DAO {
    29 references
    protected PDO $db;

    /**
     * Initialise le DAO avec l'objet de connexion PDO.
     * @param PDO $db L'objet de connexion à la base de données.
     */
    0 references | 0 overrides
    public function __construct() {
        // Injection de la dépendance PDO
        $this->db = ConnectionBDD::connect();
    }
}
```

Il s'agit de ma classe mère DAO qui appelle le service de connexion à la base de données.

```

4 references | 0 implementations
class EditionDAO extends DAO implements IEditionReader {

    // Read

    3 references | 0 overrides
    public function getActive(): Edition {
        try {
            $query = $this->db->query(query: "SELECT * FROM edition WHERE active = 1");
            $editions = $query->fetchAll(mode: PDO::FETCH_ASSOC);

            // Aucune édition active
            if (count(value: $editions) === 0) {
                throw new Exception(message: "Aucune édition active n'est définie.");
            }

            // Plus d'une édition active → erreur grave
            if (count(value: $editions) > 1) {
                throw new Exception(message: "Plusieurs éditions actives détectées. L'état du
                système est invalide.");
            }

            // Une seule édition active → OK
            $e = $editions[0];

            return new Edition(
                id: $e["id"],
                nom: $e["nom"],
                debutNomination: (new DateTime(datetime: $e["debutNomination"])),
                debutVote: (new DateTime(datetime: $e["debutVote"])),
                debutResultat: (new DateTime(datetime: $e["debutResultat"])),
                active: $e["active"]
            );
        } catch (Exception $e) {
            error_log(message: "Erreur dans EditionDAO::getActive : " . $e->getMessage());
            throw $e;
        }
    }
}

```

On a ici EditionDAO qui permet de récupérer les données de la table edition de ma base de données. Elle implémente l'interface IEditionReader.

Vue (Page):

```
21 references | 11 implementations
abstract class Page {

    2 references
    protected string $title;

    0 references
    protected $content;

    1 reference
    protected SessionManager $session;

    8 references | 8 overrides
    public function __construct(string $title) {
        $this->title = "TopTracks - " . $title;
        $this->session = SessionManager::getInstance();
    }

    1 reference | 0 overrides
    protected function renderHeader(): void {
        $header = new Header();
        $header->render();
    }

    1 reference | 0 overrides
    protected function renderFooter(): void {
        $footer = new Footer();
        $footer->render();
    }

    1 reference | 0 overrides
    protected function renderMessage(): void {
        $session = SessionManager::getInstance();
        $successMessage = $session->getSuccessMessage();
        $errorMessage = $session->getErrorMessage();
        if ($successMessage) {
            echo '<div class="message success-message">' . htmlspecialchars(string: $successMessage);
        }
        if ($errorMessage) {
            echo '<div class="message error-message">' . htmlspecialchars(string: $errorMessage);
        }
    }

    1 reference | 11 overrides
    abstract protected function renderContent();

    11 references | 0 overrides
    public function render(): void {
        ?>
        <!DOCTYPE html>
        <html lang="fr">
```

Il s'agit de ma classe mère page qui gère l'affichage du titre, du header et du footer de chaque page.

```

class PageAccueil extends Page {
    3 references
    private PhaseVote $phase;
    2 references
    private array $categories;

    8 references | 0 overrides | prototype
    public function __construct($title, $phase, $categories) {
        parent::__construct(title: $title);
        $this->phase = $phase;
        $this->categories = $categories;
    }

    1 reference | 0 overrides | prototype
    protected function renderContent(): void {
        ?>
        <section class="content">
        <?php
        switch ($this->phase) {
            case PhaseVote::PreVote:
                echo "<h2>Bienvenue sur Toptracks</h2>";
                echo "<p>Le prochain vote commencera le ...</p>";
                echo '<a href="?page=inscription">Créer un compte pour etre pres à voter</a>';
                break;
            case PhaseVote::Vote1:
                echo "<h2>Phase Vote 1</h2>";
                $this->AfficherCategories();
                break;
            case PhaseVote::Vote2:
                echo "<h2>Phase Vote 2</h2>";
                $this->AfficherCategories();
                break;
            case PhaseVote::Resultats:
                echo "<h2>Résultats</h2>";
                $this->AfficherCategories();
                break;
            default:
                echo "<p>Phase inconnue.</p>";
        }
        ?>
        </section>
        <?php
    }
}

```

Voici ma pageAccueil on y complète le constructeur de la classe Page mère avec les élément transmit depuis le controller et la fonction renderContent() qui affiche le contenu de la page.

Vue(Composant):

```
<?php
namespace vue\composant;

19 references | 10 implementations
abstract class Composant {
    2 references
    protected string $className;

    8 references | 8 overrides
    public function __construct(string $className = "composant") {
        $this->className = $className;
    }

    1 reference | 10 overrides
    abstract protected function renderContent();

    6 references | 0 overrides
    public function render(): void {
        echo "<div class='{ $this->className }'>";
        $this->renderContent();
        echo "</div>";
    }
}
```

Il s'agit de ma classe mere Composant qui gère le nom de classe de ceux-ci.

```
<?php
namespace vue\composant;

require_once 'vue/composant/Composant.php';
require_once 'service/Enum.php';

use service\PhaseVote;
use vue\composant\Composant;

2 references | 0 implementations
class CarteCategorie extends Composant {
    2 references
    private $phase;
    6 references
    private $categorie;

    8 references | 0 overrides | prototype
    public function __construct($phase, $categorie) {
        parent::__construct(className: "carte carte-categorie");
        $this->phase = $phase;
        $this->categorie = $categorie;
    }

    1 reference | 0 overrides | prototype
    protected function renderContent(): void {
        echo '<a href="' . $this->getUrl() . '">';
        ?>
        <h3><?php echo $this->categorie->getNom(); ?></h3>
        <p><?php echo $this->categorie->getDescription(); ?></p>
        </a>
    }

    1 reference
    private function getUrl(): string {
        switch ($this->phase) {
            case PhaseVote::Vote1:
                return "?page=proposition&categorie=" . $this->categorie->getId();
            case PhaseVote::Vote2:
                return "?page=vote&categorie=" . $this->categorie->getId();
            case PhaseVote::Resultats:
                return "?page=resultat&categorie=" . $this->categorie->getId();
            default:
                return "#";
        }
    }
}
```

Tout comme pour la PageAccueil CarteCategorie complete le constructeur de sa classe mère avec les données transmises par le controller. On complète également la fonction renderContent() qui affiche le contenu de notre composant.

5. Conclusion:

Les difficultés rencontrées:

L'une des principales difficultés rencontrées a été la modélisation complète de l'application avant le développement. Faire des diagrammes cohérents et anticiper l'ensemble des interactions s'est révélé complexe, notamment parce que certaines contraintes n'apparaissent qu'au moment de l'implémentation. De plus, maintenir ces modélisations en parallèle du développement demandait une organisation importante et c'est relevé particulièrement complexe.

Les principaux objectifs ont été atteints:

- Le vote est bien anonyme grâce à la structure de notre base de données
- L'unicité du vote est garantie du moins pour chaque compte utilisateur
- Le vote est séparé en 4 phases distinctes: La période avant le vote, la phase de proposition, la phase de vote, et la phase de résultat
- L'affichage des résultats est totalement automatisé, il est impossible pour un utilisateur d'accéder à ceux-ci avant la phase de résultat.
- Un espace social est disponible pour chaque proposition (des posts de candidat et commentaire)
- Les rôles et sessions sont correctement gérés
- Les principes SOLID sont majoritairement respectés

Les limites de l'application

- La gestion des dépendances pourrait être améliorée pour respecter pleinement les principes SOLID
- La gestion et la modération des éléments n'ont pas d'interface propre elle doit être effectuée directement dans la base de données
- Le passage d'une édition à une autre n'a pas de processus automatisé elle doit être effectuée en base de données
- Certains éléments manquent d'ergonomie et d'accessibilité.
- Le manque de tests unitaires et d'intégration
- Le système de candidat n'est pas pleinement implémenté, ils devraient être automatiquement attribués aux propositions les concernant.

Les opportunités d'évolution

- Mettre en place une meilleure gestion des dépendances et faire cela de manière modulaire pour qu'elles puissent être réutilisées dans différents projets.

- Mettre en place une interface administrateur pour automatiser la programmation des phases et édition mais également simplifier la modération.
- Ajouter un système de like à l'espace interaction
- Rendre l'application responsive pour tous les appareils.